6.6 Subclassing API (Lập trình Hướng đối tượng)

- Cả Sequential và Functional API đều là "Tuyên bố" (Declarative). Nghĩa là mô hình là đồ thị tĩnh.
- Nếu bạn cần một mô hình với kiến trúc "Động", có vòng lặp, có `if/else` hoạt động linh hoạt, hãy dùng **Subclassing API**.

```
class WideAndDeepModel(keras.models.Model):
    def __init__(self, units=30, activation="relu", **kwargs):
        super().__init__(**kwargs)
        self.hidden1 = keras.layers.Dense(units, activation=activation)
        self.hidden2 = keras.layers.Dense(units, activation=activation)
        self.main_output = keras.layers.Dense(1)

    def call(self, inputs): # Dinh nghia cach truyen tien tai day
        hidden1 = self.hidden1(inputs)
        hidden2 = self.hidden2(hidden1)
        concat = keras.layers.concatenate([inputs, hidden2])
        return self.main_output(concat)
```

7.1 Lưu và Khôi phục Mô hình

- Quá trình huấn luyện Mạng Nơ-ron tốn rất nhiều thời gian (Có thể vài ngày). Bạn cần lưu lại!
- Keras lưu mô hình theo định dạng HDF5 (.h5). Nó chứa toàn bộ Trọng số, Cấu trúc, Optimizer.

```
# Luu model
model.save("my_keras_model.h5")

# Phuc hoi model
model = keras.models.load_model("my_keras_model.h5")
```

7.2 Callbacks: Trợ lý huấn luyện

- Keras cho phép gọi các hàm **Callback** ở mỗi cuối Epoch hoặc cuối Batch.
- **ModelCheckpoint**: Tự động lưu mô hình mỗi khi nó "Đạt điểm cao nhất" trên tập Validation (Tránh mất mô hình tốt nhất nếu bị sập nguồn).

```
checkpoint_cb = keras.callbacks.ModelCheckpoint("my_best_model.h5",
                                                save_best_only=True)
history = model.fit(X_train, y_train, epochs=10,
                    validation_data=(X_valid, y_valid),
                    callbacks=[checkpoint_cb])
```

7.3 Dừng sớm (Early Stopping)

- Không ai biết cần đặt epochs bao nhiêu là vừa. Đặt lớn quá sẽ gây Overfitting.
- **EarlyStopping**: Tự động "Rút phích cắm" dừng huấn luyện nếu nhận thấy Loss trên tập Validation không còn cải thiện.

```
early_stopping_cb = keras.callbacks.EarlyStopping(patience=10,
                                                  restore_best_weights=True)
# patience=10 nghia la doi them 10 epoch xem co hy vong khong.
history = model.fit(X_train, y_train, epochs=1000, # Dat epoch 1000 cho thoai mai
                    validation_data=(X_valid, y_valid),
                    callbacks=[checkpoint_cb, early_stopping_cb])
```

7.4 TensorBoard: Công cụ Giám sát tối thượng

- TensorFlow cung cấp **TensorBoard**, một giao diện đồ họa siêu cấp để theo dõi quá trình huấn luyện trực tiếp.
- Nó vẽ đồ thị Loss, Accuracy theo thời gian thực.
- Hiển thị sơ đồ đồ thị kết nối Lớp (Graph).
- Cực kỳ hữu ích để gỡ lỗi các mạng khổng lồ.
- Ghi log bằng cách truyền `keras.callbacks.TensorBoard(log_dir="logs")` vào `callbacks`.

8.1 Tinh chỉnh Siêu Tham số (Hyperparameters)

- Mạng Nơ-ron có độ linh hoạt tuyệt đối. Đồng nghĩa với việc có hàng tá thông số cần tinh chỉnh!
- Làm sao để biết mạng nên có 2 lớp hay 100 lớp ẩn?
- Phương pháp cơ bản: Sử dụng `GridSearchCV` hoặc `RandomizedSearchCV`.
- Tuy nhiên, với Mạng Sâu, `RandomizedSearch` hiệu quả hơn nhiều. Bạn cũng có thể dùng công cụ **Keras Tuner**.

8.2 Số lượng Lớp Ẩn (Hidden Layers)

- Mạng có 1 lớp ẩn với cấu trúc rộng theo lý thuyết có thể xấp xỉ bất kỳ hàm nào. Nhưng rất tốn tham số!
- **Mạng Sâu (Nhiều lớp ẩn):** Có hiệu suất tham số lớn hơn (parameter efficiency). Nó học cách phân cấp dữ liệu.
- Ví dụ nhận diện mặt: Lớp 1 học vẽ đoạn thẳng, lớp 2 vẽ thành mắt/mũi/miệng, lớp 3 gộp thành khuôn mặt.
- Kinh nghiệm: Hãy bắt đầu với 1 hoặc 2 lớp ẩn (Nó sẽ hoạt động tốt cho đa số bài toán). Đối với ảnh phức tạp, cần hàng chục lớp.

8.3 Số lượng Nơ-ron trên mỗi Lớp Ẩn

- Cách thiết kế cổ điển (Hình phễu): Lớp đầu có nhiều nơ-ron (Vd: 300), lớp tiếp theo giảm dần (100), rồi (50)... nhằm tinh lọc dữ liệu.
- Tuy nhiên, nghiên cứu chỉ ra rằng **đặt số nơ-ron bằng nhau** ở mọi lớp ẩn (ví dụ: tất cả 150) cho ra kết quả tương tự, thậm chí tốt hơn, mà lại giảm bớt số tham số bạn phải nghĩ cách cấu hình.
- Mẹo vặt chiến thuật: Hãy chọn mô hình có số lớp và số nơ-ron "thừa thãi" nhiều hơn mức cần thiết, sau đó sử dụng **Early Stopping** và **Dropout** để ngăn nó Overfitting.

8.4 Tốc độ học (Learning Rate)

- Tốc độ học (`lr`) là siêu tham số **QUAN TRỌNG NHẤT**. Nếu chỉ có thời gian tinh chỉnh 1 tham số, hãy tinh chỉnh cái này!
- Nếu `lr` quá lớn: Mạng sẽ nhảy qua lại không thể hội tụ (Diverge).
- Nếu `lr` quá nhỏ: Mất rất nhiều ngày tháng để huấn luyện.
- Giá trị khởi điểm kinh điển: 10^{-3} hoặc 10^{-4}.

8.5 Kích thước Lô (Batch Size) & Optimizer

- **Batch Size:** Thường từ 32 đến 128. Batch Size quá to sẽ huấn luyện nhanh nhờ GPU nhưng có nguy cơ rơi vào tối ưu cục bộ. Batch Size nhỏ tốn CPU, tốn thời gian nhưng mang tính "nhảy múa" cao, dễ thoát hố sâu cục bộ.
- **Optimizer:** Thuật toán **Adam** Optimizer (Sẽ học ở chương 11) được chứng minh tốt hơn nhiều so với **SGD** cổ điển và đang được dùng mặc định ở hầu hết bài toán Học sâu.

HỎI & ĐÁP